Statistiques et Optimisation pour Machine Learning

Zouhir ERRAJI

Objectifs du module

- Comprendre les bases probabilistes du Machine Learning
- Maîtriser les modèles statistiques fondamentaux
- Comprendre l'optimisation des fonctions de coût

Expérience aléatoire

Définition

Une expérience aléatoire est une expérience dont :

- On connaît les résultats possibles
- On ne peut pas prévoir le résultat exact à l'avance

Exemples :

- Lancer une pièce
- Lancer un dé
- Mesurer la température demain
- Prédire si un email est Spam

Espace des résultats

On note Ω l'ensemble des résultats possibles.

Exemples :

- Pièce : $\Omega = \{Pile, Face\}$
- Dé : $\Omega = \{1, 2, 3, 4, 5, 6\}$
- Email : $\Omega = \{Spam, NonSpam\}$

Espace probabilisé

Un espace probabilisé est un triplet :

$$(\Omega, \mathcal{F}, P)$$

où :

- Ω : ensemble des issues= L'univers des possibilté
- $\mathcal{F}$: ensemble des événements
- P : mesure de probabilité

Les événements

Un événement est un sous-ensemble de Ω.
Exemple : dé

$$A = \{2, 4, 6\}$$

événement : "obtenir un nombre pair"

Propriétés de la probabilité

La probabilté est une fonction $P : \mathcal{F} \to [0, 1]$ vérifie :

1. $P(A) \geq 0$
2. $P(\Omega) = 1$
3. Si $A \cap B = \emptyset$ alors
$$P(A \cup B) = P(A) + P(B)$$

Probabilité conditionnelle

Définition
Si $P(B) > 0$:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

$P(A|B)$ = probabilité de A sachant que B est (sera) réalisé. On restreint l'univers à B.
Donc l'intersection s'exprime :

$$P(A \cap B) = P(A|B)P(B)$$

Exemple

Dé équilibré.
A : nombre pair B : nombre > 3

$$A = \{2, 4, 6\} \quad B = \{4, 5, 6\}$$

$$P(A|B) = \frac{2}{3}$$

Indépendance

Deux événements A et B sont indépendants si :

$$P(A \cap B) = P(A)P(B)$$

Équivalent à :

$$P(A|B) = P(A)$$

Indépendance = l'information sur B ne change rien sur A.

Exemple d'indépendance

Deux lancers de pièce.
A : premier lancer = Pile B : deuxième lancer = Pile

$$P(A \cap B) = \frac{1}{4}$$

$$P(A)P(B) = \frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$$

Donc indépendants.

Probabilité composée : définition

La **probabilité composée** exprime la probabilité de l'intersection de deux événements A et B :

$$P(A \cap B) = P(A) \cdot P(B|A) = P(B) \cdot P(A|B)$$

où $P(B|A)$ est la **probabilité conditionnelle** de B sachant A.
Exemple Un sac contient 3 boules rouges et 2 boules bleues. On tire deux boules successivement sans remise.

$$P(\text{Rouge puis Bleue}) = P(R_1) \cdot P(B_2|R_1) = \frac{3}{5} \cdot \frac{2}{4} = \frac{3}{10}$$

Plus généralement :

$$P(A_1 \cap \cdots \cap A_n) = P(A_1) \prod_{i=2}^{n} P(A_i|A_1, \ldots, A_{i-1})$$

Théorème des probabilités totales

Soit $\{C_1, C_2, \ldots, C_n\}$ une **partition de l'univers** Ω, c'est-à-dire :

$$C_i \cap C_j = \emptyset \text{ pour } i \neq j \quad \text{et} \quad \bigcup_{i=1}^{n} C_i = \Omega$$

Alors, pour tout événement A :

$$\boxed{P(A) = \sum_{i=1}^{n} P(A \cap C_i) = \sum_{i=1}^{n} P(A|C_i)P(C_i)}$$

Exemple : Un produit peut provenir de 3 usines : C_1, C_2, C_3, avec $P(C_1) = 0.4$, $P(C_2) = 0.35$, $P(C_3) = 0.25$. Probabilité de défaut A : $P(A|C_1) = 0.01$, $P(A|C_2) = 0.02$, $P(A|C_3) = 0.03$

$$P(A) = 0.4 \cdot 0.01 + 0.35 \cdot 0.02 + 0.25 \cdot 0.03 = 0.019$$

Théorème de Bayes

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Où :

- $P(A)$: probabilité a priori
- $P(A|B)$: probabilité a posteriori
- $P(B)$: constante de normalisation

Exemple : Test médical

Une maladie touche 1% de la population :

$$P(M) = 0.01$$

Le test :

- Sensibilité : $P(+ \mid M) = 0.9$
- Faux positif : $P(+ \mid \bar{M}) = 0.05$

Question :

$$P(M \mid +) \ ?$$

Étape 1 : Calcul de $P(+)$

$$P(+) = P(+ \mid M)P(M) + P(+ \mid \bar{M})P(\bar{M})$$

$$= 0.9 \times 0.01 + 0.05 \times 0.99$$

$$= 0.009 + 0.0495 = 0.0585$$

Étape 2 : Application de Bayes

$$P(M \mid +) = \frac{P(+ \mid M)P(M)}{P(+)}$$

$$= \frac{0.9 \times 0.01}{0.0585} = \frac{0.009}{0.0585} \approx 0.154$$

$$\boxed{P(M \mid +) \approx 15.4\%}$$

Cas discret général

Pour une partition $(C_1, \ldots, C_K)$:

$$P(C_k|X) = \frac{P(X|C_k)P(C_k)}{\sum_{j=1}^{K} P(X|C_j)P(C_j)}$$

Formule centrale du Machine Learning probabiliste.

Formule de décision

$$P(C_k|X) = \frac{P(X|C_k)P(C_k)}{P(X)}$$

Comme $P(X)$ est constant :

$$\hat{y} = \arg\max_k P(X|C_k)P(C_k)$$

Hypothèse d'indépendance conditionnelle

Pour $X = (X_1, \ldots, X_d)$
Naive Bayes suppose :

$$P(X|C_k) = \prod_{i=1}^{d} P(X_i|C_k)$$

Hypothèse forte mais efficace en pratique.

Règle de décision finale

$$\hat{y} = \arg\max_k P(C_k) \prod_{i=1}^{d} P(X_i|C_k) := \arg\max_k Scor(C_k)$$

Exemple : Classification Spam

Classes :

$$C_1 = \text{Spam}, \quad C_2 = \text{Non Spam}$$

Caractéristiques :

$$X_1 = \text{"gratuit"}$$

$$X_2 = \text{"promotion"}$$

Nous voulons répondre à la question : **Ce message est-il un Spam ou non ?**

Calcul des probabilités

La base d'apprentissage (la base d'entraînement) : Dans une boite : P(Spam)=0.4 et P(NonSpam)=0.6. On compte le nombre de repetition des mot "gratuit" et "promotion" dans toute la boite, si on trouve par ex :

$$P(\text{gratuit}|Spam) = 0.8$$

$$P(\text{promotion}|Spam) = 0.7$$

Alors :

$$Score(Spam) = 0.4 \times 0.8 \times 0.7 = 0.224$$

Calcul pour la classe Non-Spam

et

$$P(\text{gratuit}|NonSpam) = 0.1$$

$$P(\text{promotion}|NonSpam) = 0.2$$

Alors :

$$Score(NonSpam) = 0.6 \times 0.1 \times 0.2 = 0.012$$

Comparaison des scores

$$Score(Spam) > Score(NonSpam)$$

Alors :

$$\hat{y} = Spam$$

Ainsi, Le classifieur Naive Bayes prédit que : Le message est Spam.
Exr ...

Résumé :

- Problème : classification supervisée
- Objectif : prédire la classe Y à partir des variables $X = (X_1, \ldots, X_d)$

$$\hat{y} = \arg\max_{y \in \mathcal{Y}} P(Y = y \mid X = x)$$

$$= \arg\max_{y} P(Y = y) \prod_{j=1}^{d} P(X_j \mid Y = y)$$

Algorithme (cas discret)

Phase d'apprentissage :

- Estimer $P(Y = y)$
- Estimer $P(X_j = x_j \mid Y = y)$

Phase de prédiction :

- Calculer le score pour chaque classe
- Choisir la classe maximisant le score

Types de Naive Bayes

- **Bernoulli NB** : variables binaires
- **Multinomial NB** : texte (fréquence de mots)
- **Gaussian NB** : variables continues

Cas Gaussien :

$$P(X_j \mid Y = y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x_j - m_y)^2}{2\sigma_y^2}\right)$$

2) Variables aléatoires

- Variables discrètes et continues
- Fonction de répartition
- Espérance et variance
- Lois classiques (Bernoulli, Binomiale, Normale)

Définition : Variable aléatoire

Idée intuitive
Une variable aléatoire transforme un résultat abstrait en nombre.
Définition
Une variable aléatoire X est une application :

$$X : \Omega \to \mathbb{R}$$

Elle associe à chaque issue $\omega \in \Omega$ un nombre réel.

Exemple simple

Lancer un dé :

$$\Omega = \{1, 2, 3, 4, 5, 6\}$$

On définit :

$$X(\omega) = \omega^2$$

Ici X est la valeur obtenue.

Autre exemple

Classification email :

$$\Omega = \{succs, chec\}$$

On code :

$$X(succs) = 1, \quad X(chec) = 0$$

Le ML travaille toujours avec des variables numériques.

Variable aléatoire discrète

X est discrète si elle prend un nombre fini ou dénombrable de valeurs qu'on note $X(\Omega) = \{x_1, ..., x_i, ...\}$.
On définit la loi de probabilité :

$$P(X = x_i) == \text{La probabilté que} \quad X \quad \text{egale à} \quad x_i$$

avec :

$$\sum_i P(X = x_i) = 1$$

Dans l'exemple precédent $X(\Omega) = \{0, 1\}$, et la loi de probabilté est P(X = 1)= p (probabilté de succès) et P(X = 0)= ...

Variable Continue : Densité de probabilité

Une fonction f :$\mathbb{R} \to \mathbb{R}$. est dite **densité de probabilité** si les conditions suivantes sont vérifiées :

$$f(x) \geq 0 \qquad \forall x \in \mathbb{R}$$

$$\int_{-\infty}^{+\infty} f(x), dx = 1$$

Pour tout intervalle ([a,b]$\subset \mathbb{R}$),

$$P(a \leq X \leq b) = \int_a^b f(x), dx$$

Variable aléatoire continue

La va X est dite continue si elle admet une densité f telle que :
Pour tout intervalle ([a,b]$\subset \mathbb{R}$), :

$$P(a \leq X \leq b) = \int_a^b f(x)\, dx$$

Espérance

Espérance (discret)

$$\mathbb{E}[X] = \sum_i x_i P(X = x_i)$$

Espérance (continu)

$$\mathbb{E}[X] = \int_{-\infty}^{+\infty} x f(x)\, dx$$

Interprétation : moyenne théorique.

Variance

La variance mesure la dispersion autour de la moyenne.

$$\mathrm{Var}(X) = \mathbb{E}\left[(X - \mathbb{E}[X])^2\right]$$

Formule pratique :

$$\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$$

Interprétation :

- Variance faible $\rightarrow$ données concentrées
- Variance grande $\rightarrow$ données dispersées

Fondamental pour comprendre le surapprentissage.

Linéarité de l'espérance

Soient X et Y deux variables aléatoires et $a, b \in \mathbb{R}$.

Propriété de linéarité

$$E(aX + bY) = aE(X) + bE(Y)$$

- Cette propriété est valable **même si X et Y ne sont pas indépendantes**.
- Elle se généralise à plusieurs variables aléatoires :

$$E\left(\sum_{i=1}^{n} a_i X_i\right) = \sum_{i=1}^{n} a_i E(X_i)$$

Propriétés de la variance

Soit X une variable aléatoire et $a, b \in \mathbb{R}$.

Variance d'une transformation linéaire

$$Var(aX + b) = a^2 Var(X)$$

Variance d'une somme

$$Var(X + Y) = Var(X) + Var(Y)$$

si X et Y sont indépendantes.

L'écart type

$$\sigma_X = \sqrt{\mathrm{Var}(X)}$$

Covariance

Pour deux variables X et Y :

$$\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}(X))(Y - \mathbb{E}(Y)]$$

Mesure la **variation conjointe** entre X et Y.

- $Cov(X, Y) > 0$: les variables évoluent dans le même sens
- $Cov(X, Y) < 0$: les variables évoluent en sens opposé
- $Cov(X, Y) = 0$: absence de relation linéaire

Corrélation

$$\rho(X, Y) = \frac{\mathrm{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

Elle mesure la **force de la relation linéaire** entre X et Y

- $\rho = 1$: corrélation parfaite
- $\rho = 0$: non corrélées

Rôle en Machine Learning

- Détection des **relations entre variables**
- **Sélection de variables** (feature selection)
- Réduction de dimension (ex : **PCA**)
- Analyse de la **redondance entre features**

Exemples concrets en Machine Learning

- **Détection des relations entre variables**
  Exemple : dans un modèle de prédiction du **prix des maisons**, la variable *surface* est souvent fortement corrélée avec le *prix*. Une corrélation positive indique que lorsque la surface augmente, le prix augmente.
- **Sélection de variables (Feature Selection)**
  Exemple : pour prédire le **risque de diabète**, on peut analyser la corrélation entre les variables (âge, IMC, taux de glucose). Les variables ayant une forte corrélation avec la cible sont conservées dans le modèle.

- **Réduction de dimension (PCA)**
  Exemple : dans la **reconnaissance d'images**, une image peut contenir des milliers de pixels. La méthode PCA utilise la matrice de covariance pour transformer ces variables en quelques composantes principales qui résument l'information importante.

Exemple concrèt

Considérons les données utilisé pour prédire le prix d'une maison en fonction de sa surface.

$$X = \text{surface } (\text{m}^2), \qquad Y = \text{prix}$$

Données observées :

X	Y
50	150
60	180
80	240
100	300

$$E(X) = \frac{50 + 60 + 80 + 100}{4} = 72.5$$

$$E(Y) = \frac{150 + 180 + 240 + 300}{4} = 217.5$$

$$Cov(X, Y) = \frac{1}{n}\sum_{i=1}^{n}(x_i - E(X))(y_i - E(Y))$$

Détail du calcul :

$$(50 - 72.5)(150 - 217.5) = (-22.5)(-67.5) = 1518.75$$
$$(60 - 72.5)(180 - 217.5) = (-12.5)(-37.5) = 468.75$$
$$(80 - 72.5)(240 - 217.5) = (7.5)(22.5) = 168.75$$
$$(100 - 72.5)(300 - 217.5) = (27.5)(82.5) = 2268.75$$

$$Cov(X, Y) = \frac{4425}{4} = 1106.25$$

Covariance positive $\Rightarrow$ relation croissante.

$$\sigma_X = \sqrt{\frac{1}{n}\sum(x_i - E(X))^2}$$

$$(50 - 72.5)^2 = (-22.5)^2 = 506.25$$
$$(60 - 72.5)^2 = (-12.5)^2 = 156.25$$
$$(80 - 72.5)^2 = (7.5)^2 = 56.25$$
$$(100 - 72.5)^2 = (27.5)^2 = 756.25$$

$$\sigma_X = \sqrt{\frac{1475}{4}} = \sqrt{368.75} \approx 19.20$$

Analogue :

$$\sigma_Y = \sqrt{\frac{13275}{4}} = \sqrt{3318.75} \approx 57.61$$

Calcul de la corrélation

$$\rho(X, Y) = \frac{Cov(X, Y)}{\sigma_X \sigma_Y} = \frac{1106.25}{19.20 \times 57.61}$$

$$19.20 \times 57.61 \approx 1106.11$$

$$\rho(X, Y) \approx \frac{1106.25}{1106.11} \approx 1.0001$$

Soit $\rho(X, Y) \approx 1$: les variables sont fortement corrélées.
Une forte corrélation signifie que la variable X est utile pour prédire Y. Elle sera donc sélectionnée comme **feature importante** dans un modèle de régression.

Lois Classiques : Loi de Bernoulli

Une variable aléatoire X suit une loi de Bernoulli de paramètre p si :

$$X \in \{0, 1\}$$

avec :

$$P(X = 1) = p \quad P(X = 0) = 1 - p$$

On note :

$$X \sim \mathcal{B}(p)$$

Son Espérance et Variance

$$\mathbb{E}[X] = p$$
$$\mathrm{Var}(X) = p(1 - p)$$

Interprétation Machine Learning

Classification binaire :

$$Y \in \{0, 1\}$$

On modélise :

$$Y \sim \mathcal{B}(p(x))$$

La régression logistique consiste à modéliser :

$$p(x) = P(Y = 1 | X = x_i)$$

Loi Binomiale

Soit X le nombre de succès dans une série d'expériences indépendantes de Bernoulli $\mathcal{B}(p)$
On a $X(\Omega) = \{0, 1, ..., n\}$
On dit que X la loi binomial de paramétre n : nombre de répétitions ou d'essais et p : probabilité de succès à chaque essai
On note

$$X \sim \mathcal{B}(n, p)$$

Sa loi de Probabilité est :

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

Moments de loi Binomial

Si

$$X \sim \mathcal{B}(n, p)$$

Alors

$$\mathbb{E}[X] = np$$

$$\mathrm{Var}(X) = np(1-p)$$

Définition de la loi normale

Une variable aléatoire réelle X suit une loi normale (ou gaussienne) de paramètres μ (espérance) et σ^2 (variance) si sa densité de probabilité est donnée par :

$$f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}, \quad x \in \mathbb{R}.$$

On note $X \sim \mathcal{N}(\mu, \sigma^2)$.

- μ est un paramètre de position (centre de la courbe).
- σ est un paramètre d'échelle (dispersion).
- La courbe est symétrique par rapport à μ, en forme de cloche.

Propriétés fondamentales

- Espérance : $E(X) = \mu$
- Variance : $Var(X) = \sigma^2$
- Écart-type : σ
- La loi normale est entièrement déterminée par ses deux premiers moments.
- Théorème central limite : la somme d'un grand nombre de variables aléatoires indépendantes tend vers une loi normale.

Loi normale centrée réduite

Si $X \sim \mathcal{N}(\mu, \sigma^2)$, alors la variable transformée

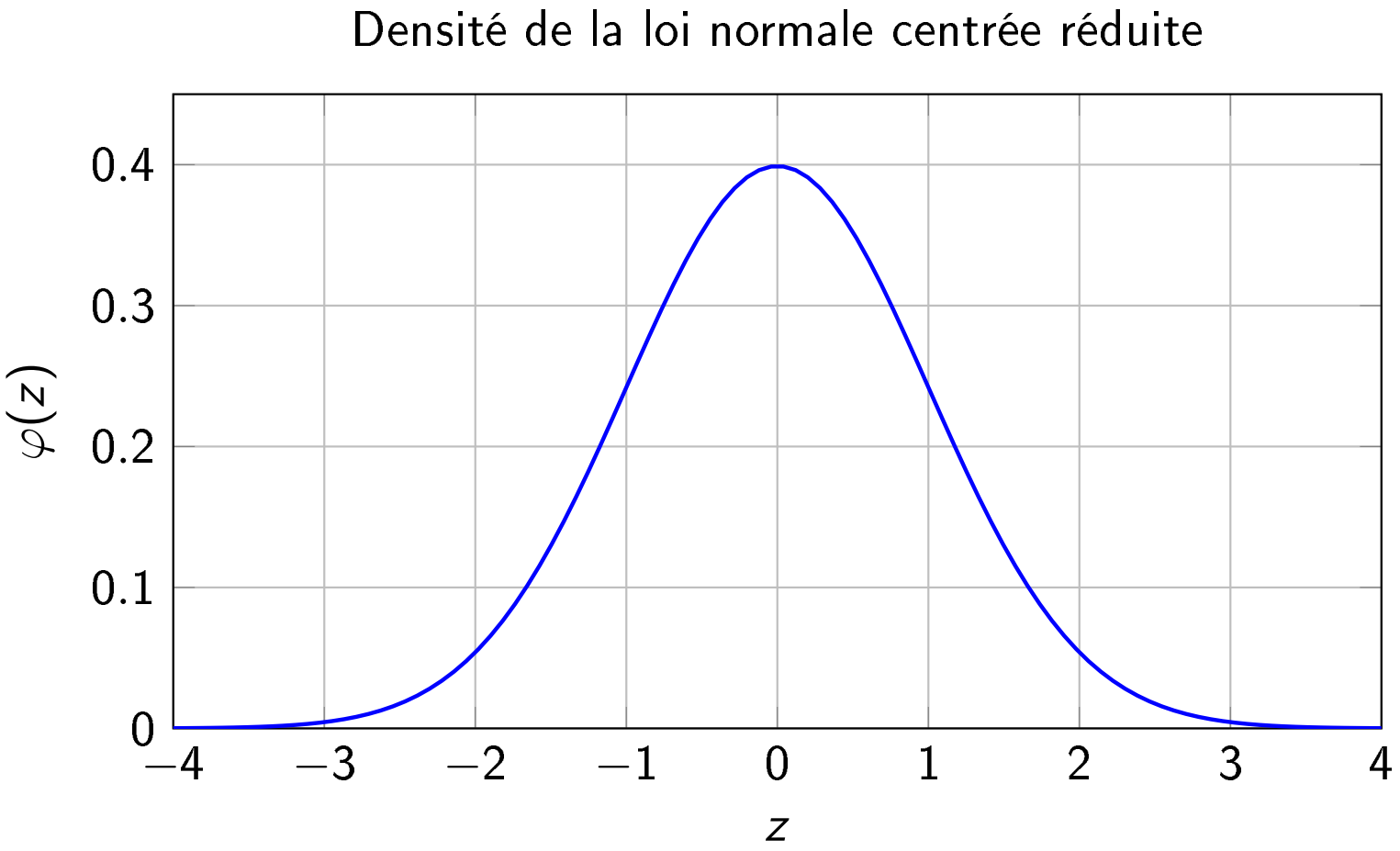
$$Z = \frac{X - \mu}{\sigma}$$

suit une loi normale d'espérance 0 et de variance 1, appelée **loi normale centrée réduite** et notée $\mathcal{N}(0, 1)$.
Sa densité est :

$$\varphi(z) = \frac{1}{\sqrt{2\pi}} e^{-\frac{z^2}{2}}, \quad z \in \mathbb{R}.$$

Cette transformation est essentielle pour ramener tous les calculs de probabilités à une seule table.

Graphe de la densité de $\mathcal{N}(0,1)$



Propriétés de la fonction de répartition Φ de $\mathcal{N}(0,1)$

Pour $Z \sim \mathcal{N}(0,1)$, $\Phi(x) = P(Z \leq x) = \int_{-\infty}^{x} \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt$.

- **Symétrie :** $\Phi(-x) = 1 - \Phi(x)$.
- $\Phi(0) = 0.5$.
- $P(Z \geq a) = 1 - \Phi(a)$.
- Quantile d'ordre α : $\Phi(z_\alpha) = \alpha$.

4) Échantillonnage

- Population vs échantillon
- Estimateur de la moyenne
- Estimateur de la variance
- Notion de biais

Lien avec le Machine Learning :

Idée clé

En ML, on n'a jamais toute la population. On travaille toujours sur un échantillon.

Population vs Échantillon

Population

Ensemble complet des individus ou observations.

Exemple :

- Tous les emails existants.
- Tous les clients d'une banque.
- Tous les étudiants d'une université.

Échantillon

Sous-ensemble extrait de la population.

Pourquoi utiliser un échantillon ?

- Impossible d'observer toute la population.
- Trop coûteux.
- Données inaccessibles.

En Machine Learning :

La base d'entraînement est un échantillon de la réalité.

Paramètre vs Estimateur

Paramètre

Quantité inconnue de la population. Exemple : moyenne réelle m.

Estimateur

Formule calculée à partir de l'échantillon. Exemple : moyenne empirique $\bar{X}$.

Définition du biais

Le biais d'un estimateur $\hat{\theta}$ est :

$$\text{Biais}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta$$

- Si biais $= 0 \rightarrow$ estimateur non biaisé.
- Sinon $\rightarrow$ estimateur biaisé.

Estimateur de la moyenne

Soit un échantillon :

$$X_1, X_2, \ldots, X_n$$

i.i.d. tels que :

$$\mathbb{E}[X_i] = m, \quad \mathrm{Var}(X_i) = \sigma^2.$$

La moyenne empirique est :

$$\bar{X} = \frac{1}{n} \sum_{i=1}^{n} X_i$$

Elle estime la moyenne réelle m.

$$\bar{X} \approx m$$

quand n est grand.

Preuve

Soit un échantillon :
La moyenne empirique est :

$$\bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i$$

Alors :

$$\mathbb{E}[\bar{X}] == \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}(X_i) = m$$

Alors l'estimateur estnon biaisé.

Estimateur de la Variance

Variance réelle :

$$\sigma^2 = \mathbb{E}[(X - m)^2]$$

Inconnue en pratique.
Estimateur empirique :

$$S^2 = \frac{1}{n-1} \sum_{i=1}^{n} (X_i - \bar{X})^2$$

Pourquoi $n - 1$?

- Correction du biais.

Cadre du problème

Si on Estime la variance avec $\frac{1}{n}$:

$$S_n^2 = \frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^2.$$

Objectif : calculer $\mathbb{E}[S_n^2]$.

Décomposition utile

On utilise l'identité :

$$\sum_{i=1}^{n}(X_i - \bar{X})^2 = \sum_{i=1}^{n}(X_i - m)^2 - n(\bar{X} - m)^2.$$

Donc :

$$S_n^2 = \frac{1}{n}\left(\sum_{i=1}^{n}(X_i - m)^2 - n(\bar{X} - m)^2\right).$$

On prend l'espérance.

Preuve

On sait que :

$$\mathbb{E}\left[\sum_{i=1}^{n}(X_i - m)^2\right] = n\sigma^2,$$

et

$$\mathbb{E}\left[(\bar{X} - m)^2\right] = \mathrm{Var}(\bar{X}) = \frac{\sigma^2}{n}.$$

Donc :

$$\mathbb{E}[S_n^2] = \frac{1}{n}\left(n\sigma^2 - n\frac{\sigma^2}{n}\right).$$

Résultat final

On obtient :

$$\mathbb{E}[S_n^2] = \frac{1}{n}\left(n\sigma^2 - \sigma^2\right) = \frac{n-1}{n}\sigma^2.$$

$$\boxed{\mathbb{E}[S_n^2] = \frac{n-1}{n}\sigma^2}$$

Donc l'estimateur avec $\frac{1}{n}$ est biaisé :

$$\mathrm{Biais}(S_n^2) = -\frac{\sigma^2}{n}.$$

C'est pourquoi on utilise :

$$\frac{1}{n-1}$$

pour obtenir un estimateur sans biais.

4.2) Maximum de vraisemblance

Idée intuitive

On observe un échantillon :

$$X_1, X_2, \ldots, X_n$$

On suppose que les données suivent une loi dépendant d'un paramètre θ.

Question :

Quelle valeur de θ rend les données les plus probables ?

Fonction de vraisemblance

Supposons que $X_1, \ldots, X_n$ sont i.i.d.
Densité : $f(x|\theta)$
La vraisemblance est :

$$L(\theta) = \prod_{i=1}^{n} f(X_i|\theta)$$

On considère les données fixes et θ variable.

Log-vraisemblance

On prend souvent le logarithme :

$$\ell(\theta) = \log L(\theta) = \sum_{i=1}^{n} \log f(X_i|\theta)$$

Pourquoi ?

- Plus simple à dériver.
- Transforme un produit en somme.

Définition

L'estimateur du maximum de vraisemblance est :

$$\hat{\theta}_{MLE} = \arg\max_{\theta} L(\theta)$$

ou équivalemment :

$$\hat{\theta}_{MLE} = \arg\max_{\theta} \ell(\theta)$$

Densité normale

$$f(x|m,\sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-m)^2}{2\sigma^2}\right)$$

La vraisemblance :

$$L(m,\sigma^2) = \prod_{i=1}^{n} f(X_i|m,\sigma^2)$$

Log-vraisemblance (σ^2 connue)

$$\ell(m) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(X_i - m)^2$$

On dérive par rapport à m :

$$\frac{d\ell}{dm} = \frac{1}{\sigma^2}\sum_{i=1}^{n}(X_i - m)$$

Condition d'optimalité

On annule la dérivée :

$$\sum_{i=1}^{n}(X_i - m) = 0$$

$$\sum X_i - nm = 0$$

Donc :

$$\hat{m}_{MLE} = \frac{1}{n}\sum_{i=1}^{n} X_i = \bar{X}$$

Estimateur de σ^2

On obtient :

$$\hat{\sigma}^2_{MLE} = \frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^2$$

Attention :

- Ici on divise par n
- Ce n'est pas l'estimateur non biaisé

Remarque importante

Le MLE peut être biaisé.
Pour la variance :

$$\frac{1}{n} \quad \text{(MLE)}$$

$$\frac{1}{n-1} \quad \text{(estimateur non biaisé)}$$

En pratique moderne :
Petit échantillon, cadre théorique $\rightarrow 1/n$
Grand échantillon, ML, optimisation $\rightarrow 1/n-1$

Résumé

- La vraisemblance mesure à quel point les données sont compatibles avec un paramètre.
- Le MLE maximise cette compatibilité.
- Pour la loi normale :

$$\hat{m} = \bar{X}$$

$$\hat{\sigma}^2 = \frac{1}{n}\sum(X_i - \bar{X})^2$$

4.3) Intervalles de confiance

- Intervalle de confiance
- Test d'hypothèse (idée générale)
- Lien avec la prise de décision

Pourquoi un intervalle ?

On a estimé :

$$\hat{m} = \bar{X}$$

Mais :

- $\bar{X}$ varie d'un échantillon à l'autre.
- Il existe une incertitude.

On veut encadrer la vraie valeur m.

Définition

Un **intervalle de confiance** (IC) pour un paramètre θ est un intervalle aléatoire :

$$IC = [L(X), U(X)]$$

tel que :

$$P(\theta \in IC) = 1 - \alpha$$

où :

- $1 - \alpha$ = niveau de confiance (95%, 99%, etc.)
- α = risque d'erreur

IC pour la moyenne (variance connue)

Soit $X_1, \ldots, X_n$ iid avec :

$$X_i \sim \mathcal{N}(m, \sigma^2)$$

On sait que :

$$\bar{X} \sim \mathcal{N}\left(m, \frac{\sigma^2}{n}\right)$$

Alors :

$$Z = \frac{\bar{X} - m}{\sigma/\sqrt{n}} \sim \mathcal{N}(0, 1)$$

Construction de l'intervalle de confiance

Donc pour $z_{\alpha/2}$ tel que

$$P(-z_{\alpha/2} \leq Z \leq z_{\alpha/2}) = 1 - \alpha, \quad Z \sim \mathcal{N}(0,1),$$

on a

$$P\left(-z_{\alpha/2} \leq \frac{\bar{X} - m}{\sigma/\sqrt{n}} \leq z_{\alpha/2}\right) = 1 - \alpha.$$

Alors :

$$P\left(\bar{X} - z_{\alpha/2}\frac{\sigma}{\sqrt{n}} \leq m \leq \bar{X} + z_{\alpha/2}\frac{\sigma}{\sqrt{n}}\right) = 1 - \alpha.$$

Donc l'intervalle de confiance est :

$$IC_{1-\alpha} = \left[\bar{X} - z_{\alpha/2}\frac{\sigma}{\sqrt{n}}, \bar{X} + z_{\alpha/2}\frac{\sigma}{\sqrt{n}}\right].$$

Interprétation correcte

Un IC à 95% signifie :

Interprétation correcte

Si on répète l'expérience un grand nombre de fois, 95% des intervalles construits contiendront le vrai paramètre.

La **marge d'erreur** est :

$$\boxed{E = z_{\alpha/2} \frac{\sigma}{\sqrt{n}}}$$

Donc :

$$IC = \bar{X} \pm E$$

Test d'hypothèse

Idée générale

Un test d'hypothèse permet de décider entre :

- H_0 : hypothèse nulle
- H_1 : hypothèse alternative

Exemple :

$$H_0 : m = m_0$$

$$H_1 : m \neq m_0$$

Règle de décision en fonction de $\bar{X}$

On teste :

$$H_0 : m = m_0 \qquad \text{contre} \qquad H_1 : m \neq m_0$$

- Si m_0 appartient à
$$\left[\bar{X} - z_{\alpha/2}\frac{\sigma}{\sqrt{n}}, \bar{X} + z_{\alpha/2}\frac{\sigma}{\sqrt{n}}\right]$$
on ne rejette pas H_0.
- Sinon, on rejette H_0.

Étapes d'un test

1. Formuler H_0 et H_1
2. Choisir le niveau α
3. Calculer la statistique de test
4. Déterminer la région critique
5. Prendre une décision

II L'optimisation

Problème d'optimisation

On considère le problème d'optimisation

$$\min_{\mathrm{x} \in C} f(\mathrm{x}),$$

où C est une partie de$\mathbb{R}^d$ s'appelle l'espace des contraintes ; f est une fonction objectif (coût, perte, vraisemblance) typiquement

$$f(\mathrm{x}) = \frac{1}{n} \sum_{i=1}^{n} \ell(\mathrm{x}; \mathrm{a}_i, b_i) + \lambda R(\mathrm{x}).$$

- ℓ : fonction de perte (quadratique, logistique, hinge...)
- R : régularisation (ℓ_2, ℓ_1, ...)
- Objectif : trouver un minimum (global ou local) pour généraliser.

Notations et rappels d'analyse

- $\nabla f(\mathrm{x})$: gradient (vecteur des dérivées partielles).
- $\nabla^2 f(\mathrm{x})$: matrice Hessienne (symétrique si $f \in C^2$).
- f est L-lisse (gradient Lipschitz) si

$$\|\nabla f(\mathrm{x}) - \nabla f(\mathrm{y})\| \leq L\|\mathrm{x} - \mathrm{y}\|, \quad \forall \mathrm{x}, \mathrm{y}.$$

- f est μ-fortement convexe si

$$f(\mathrm{y}) \geq f(\mathrm{x}) + \nabla f(\mathrm{x})^T(\mathrm{y} - \mathrm{x}) + \frac{\mu}{2}\|\mathrm{y} - \mathrm{x}\|^2, \quad \forall \mathrm{x}, \mathrm{y}.$$

Principe de Descente de gradient (GD)

La descente de gradient utilise l'approximation linéaire :

$$f(\mathrm{x}+\mathrm{d}) \approx f(\mathrm{x}) + \nabla f(\mathrm{x})^T \mathrm{d}.$$

Pour minimiser, on choisit d dans la direction de plus forte pente : $\mathrm{d} = -\eta \nabla f(\mathrm{x})$, avec $\eta > 0$ le pas.

Algorithm 1 Descente de gradient (GD)

Require: f, x_0, $\{\eta_k\}$, tolérance ε

```
k ← 0
while ‖∇f(x_k)‖ > ε do
    x_{k+1} ← x_k − η_k ∇f(x_k)
    k ← k + 1
end while
return x_k
```

Pas constant

- $\eta_k = \eta$ constant.
- Simple, mais nécessite une bonne estimation de la constante de Lipschitz.
- Si f est L-lisse, le choix $\eta = 1/L$ garantit la décroissance.
- En pratique, on utilise souvent une valeur faible (ex. 10^{-3}) et on ajuste.

Pas décroissant

Pour éviter l'oscillation et assurer la convergence, on utilise des séries divergentes :

$$\eta_k = \frac{\eta_0}{\sqrt{k+1}} \quad \text{ou} \quad \frac{\eta_0}{k+1}.$$

Théorème

Si f est convexe, L-lisse, et $\sum_{k=0}^{\infty} \eta_k = \infty$, $\sum_{k=0}^{\infty} \eta_k^2 < \infty$, alors $f(x_k) \to f(x^*)$.

Recherche linéaire

- On choisit η_k qui satisfait les conditions d'Armijo :

$$f(\mathrm{x}_k - \eta \nabla f(\mathrm{x}_k)) \leq f(\mathrm{x}_k) - c_1 \eta \|\nabla f(\mathrm{x}_k)\|^2, \quad c_1 \in (0,1).$$

- Algorithme de rétrocession (backtracking) : partir d'un η initial, diviser par β jusqu'à satisfaction.
- Garantit la convergence et évite le réglage manuel.

Cas convexe L-lisse

Théorème (Gradient descent avec pas $\eta = 1/L$)

Soit f convexe et L-lisse. Alors

$$f(x_k) - f(x^*) \leq \frac{L\|x_0 - x^*\|^2}{2k}.$$

Preuve (schéma) :

- Utiliser la descente suffisante : $f(x_{k+1}) \leq f(x_k) - \frac{1}{2L}\|\nabla f(x_k)\|^2$.
- Puis majorer la somme des gradients par la différence des distances.

Cas fortement convexe

Théorème

Si f est μ-fortement convexe et L-lisse, avec $\eta = 2/(\mu + L)$ (pas optimal), alors

$$\|x_k - x^*\|^2 \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^{2k} \|x_0 - x^*\|^2,$$

où $\kappa = L/\mu$ est le conditionnement. La convergence est linéaire (géométrique).

- Le taux de convergence dépend du conditionnement : κ grand $\Rightarrow$ convergence lente.

Gradient stochastique (SGD)

Lorsque $f(\mathrm{x}) = \frac{1}{n}\sum_{i=1}^{n} f_i(\mathrm{x})$, on estime le gradient par un sous-ensemble :

$$\mathrm{x}_{k+1} = \mathrm{x}_k - \eta_k \frac{1}{|\mathcal{B}_k|} \sum_{i \in \mathcal{B}_k} \nabla f_i(\mathrm{x}_k).$$

- $\mathcal{B}_k$est un sous-ensemble aléatoire de l'ensemble d'entraînement
- Coût par itération indépendant de n (grands jeux de données).
- Convergence en moyenne : $\mathbb{E}[f(\mathrm{x}_k)] - f(\mathrm{x}^*) \leq O(1/\sqrt{k})$ (sous conditions).
- Très populaire en deep learning.

Approximation quadratique

On approxime f au second ordre autour de x_k :

$$f(\mathrm{x}_k + \mathrm{d}) \approx f(\mathrm{x}_k) + \nabla f(\mathrm{x}_k)^T \mathrm{d} + \frac{1}{2} \mathrm{d}^T \nabla^2 f(\mathrm{x}_k) \mathrm{d}.$$

En annulant le gradient de cette quadratique par rapport à d, on obtient :

$$\nabla f(\mathrm{x}_k) + \nabla^2 f(\mathrm{x}_k) \mathrm{d} = 0 \quad \Rightarrow \quad \mathrm{d}_k = -\nabla^2 f(\mathrm{x}_k)^{-1} \nabla f(\mathrm{x}_k).$$

La direction de Newton d_k minimise la quadratique locale.

Algorithme de Newton

Algorithm 2 Newton (pure)

Require: f, x_0, tolérance ε

```
k ← 0
while ‖∇f(x_k)‖ > ε do
    Calculer H_k = ∇²f(x_k)
    Résoudre H_k d_k = −∇f(x_k)
    x_{k+1} ← x_k + d_k
    k ← k + 1
end while
return x_k
```

Convergence quadratique

Théorème

Soit $f \in C^2$, $\nabla^2 f$ Lipschitzienne au voisinage d'un minimum x^* avec $\nabla^2 f(x^*)$ définie positive. Alors il existe $\delta > 0$ tel que si $\|x_0 - x^*\| \leq \delta$, la méthode de Newton converge et

$$\|x_{k+1} - x^*\| \leq C\|x_k - x^*\|^2.$$

- Le nombre de chiffres significatifs double à chaque itération (convergence très rapide).
- Inconvénient : nécessite un point de départ suffisamment proche.

Problèmes de la méthode de Newton

1. **Coût de calcul** : $O(d^3)$ pour l'inversion de la Hessienne, $O(d^2)$ pour la construction.
2. **Hessienne non définie positive** : la direction n'est pas de descente.
3. **Sensibilité au point initial** : peut diverger si trop loin.
4. **Stockage** : matrice $d \times d$ impossible en grande dimension.

Remèdes

- **Newton modifié** : ajout d'un pas α_k par recherche linéaire :

$$x_{k+1} = x_k + \alpha_k d_k, \quad \alpha_k \text{ satisfaisant Armijo.}$$

- **Newton régularisé** : remplacer H_k par $H_k + \lambda I$ (Levenberg-Marquardt) pour garantir la positivité.
- **Quasi-Newton** : approximation de la Hessienne à partir des gradients successifs.

Régression linéaire : Nuage de points

On observe des données :

$$(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$$

Représentation graphique :

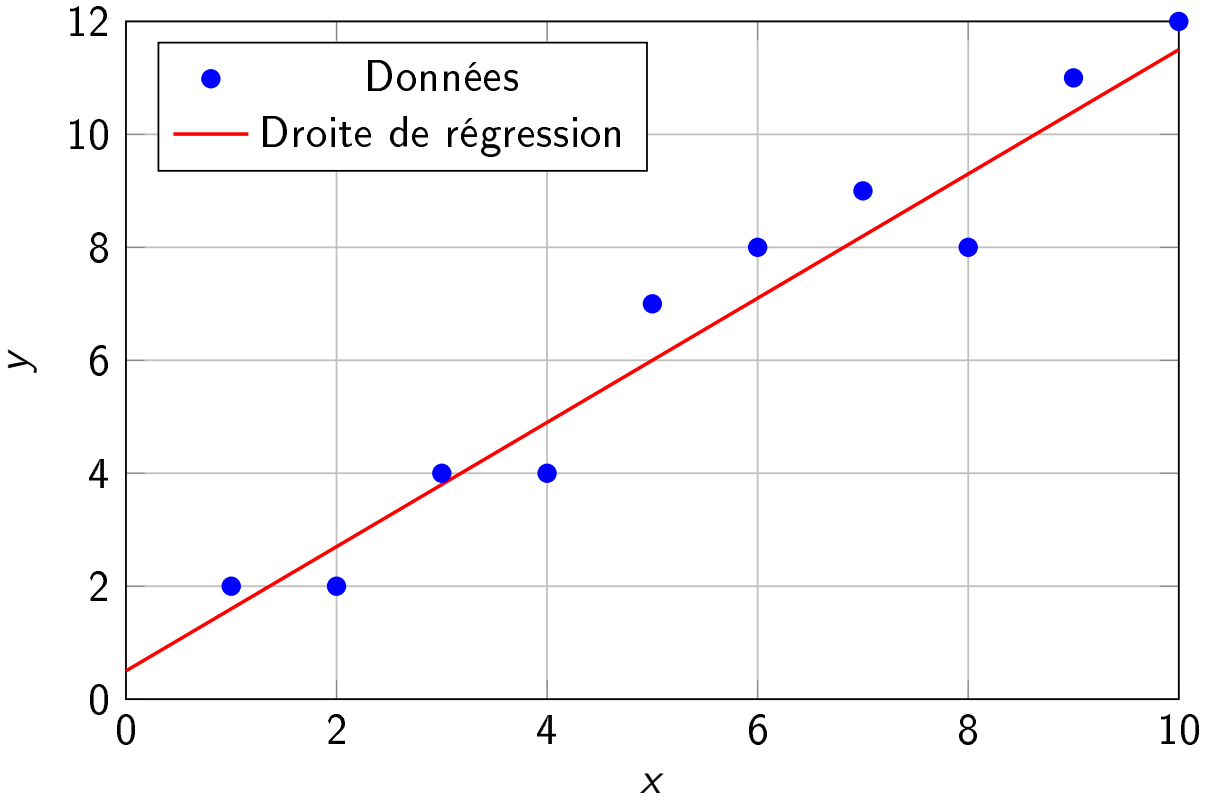
- Chaque point représente une observation.
- On observe une tendance approximativement linéaire.

Question :

Peut-on approximer ces points par une fonction affine ?

Régression linéaire

Régression linéaire : ajustement d'un nuage de points



La droite minimise la somme des carrés des résidus (distances verticales).

Modèle : Fonction affine

On cherche une fonction :

$$y = ax + b$$

qui approxime le nuage de points pour prédire des valeurs continues y (ex : ventes, prix) en analysant des tendances.

Principe des moindres carrés

On définit l'erreur au point i :

$$e_i = y_i - (aX_i + b)$$

Par le Principe des moindres carrés, On veut minimiser l'erreur globale.

$$min_{a,b} F(a,b) = \sum_{i=1}^{n} (y_i - aX_i - b)^2$$

Pourquoi le carré ?

- Évite les compensations positives/négatives.
- Donne un problème d'optimisation convexe.

Résultat final

Les valeurs exactes de a et b sont :

$$a = \frac{\mathrm{Cov}(X, Y)}{\mathrm{Var}(X)}$$

$$b = \bar{Y} - a\bar{X}$$

Petit exemple

Points :

$$(1,2),(2,3),(3,5)$$

On calcule :

$$\bar{X} = 2 \quad \bar{y} = \frac{2+3+5}{3} = \frac{10}{3}$$

Puis on applique la formule pour obtenir a et b.

logique du Machine Learning moderne

La régression linéaire consiste à :

1. Choisir une forme de fonction (affine).
2. Définir une mesure d'erreur.
3. Minimiser cette erreur.
4. Faire des prédictions.

C'est exactement la logique du Machine Learning moderne.

Régression linéaire par descente de gradient

- Modèle : $\hat{y}_i = ax_i + b$
- Coût (erreur quadratique moyenne) :

$$J(a, b) = \frac{1}{2n} \sum_{i=1}^{n} (y_i - (ax_i + b))^2$$

- Gradients :

$$\frac{\partial J}{\partial a} = -\frac{1}{n} \sum_{i=1}^{n} x_i (y_i - (ax_i + b)) \qquad \frac{\partial J}{\partial b} = -\frac{1}{n} \sum_{i=1}^{n} (y_i - (ax_i + b))$$

- Mise à jour (pas η) :

$$a \leftarrow a - \eta \frac{\partial J}{\partial a}, \quad b \leftarrow b - \eta \frac{\partial J}{\partial b}$$

Application directe : la descente de gradient tend vers le même minimum sans passer par Cov, Var... La solution par descente de gradient est préférable pour les jeux de très grands volumes.

Régression logistique : modèle

- On observe :

$$(X_i, Y_i), \quad Y_i \in \{0, 1\}$$

- Objectif :Prédire le Succès : Y=1 Sachant X :

$$P(Y = 1|X)$$

- On Suppose :

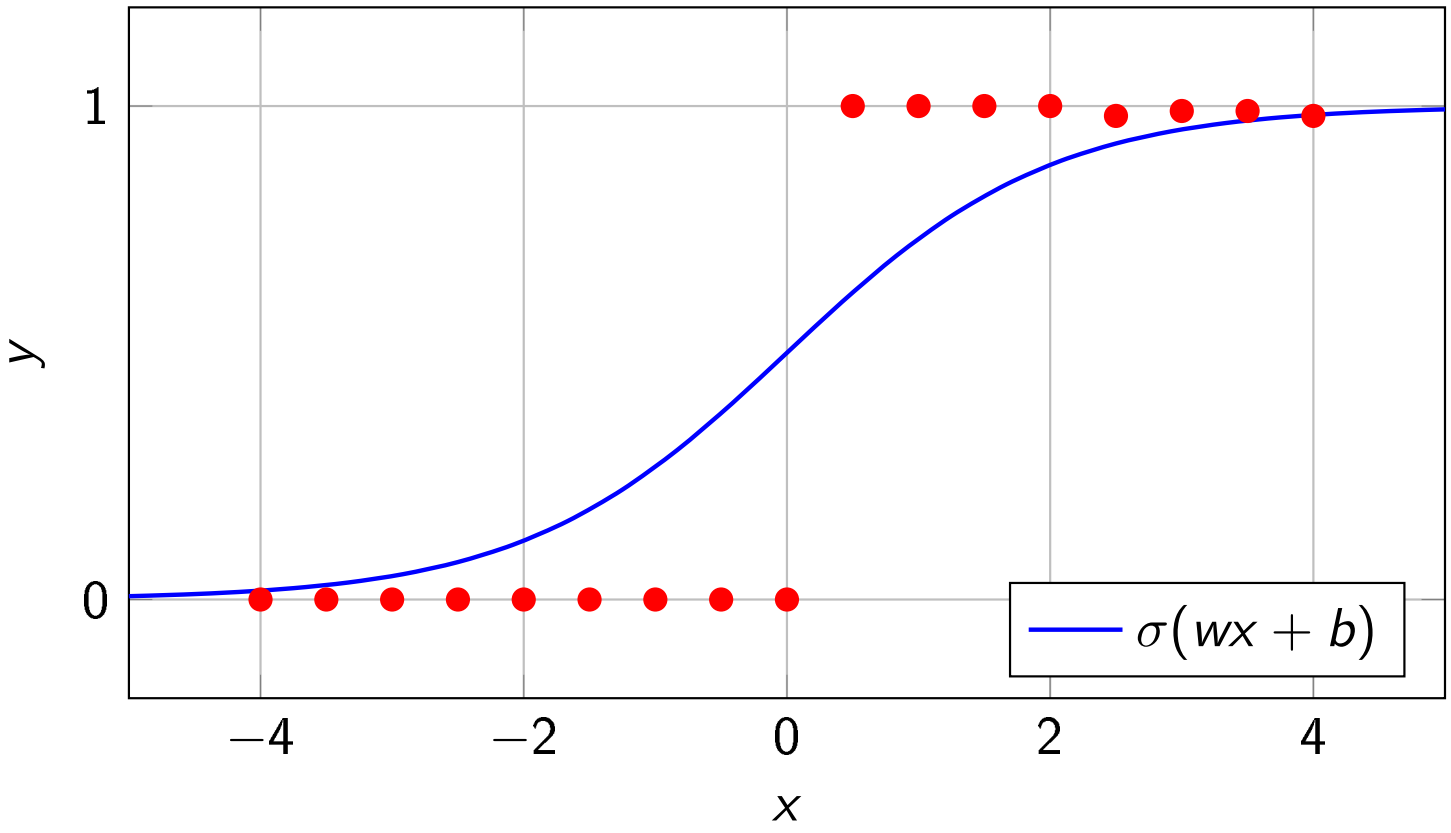
$$p(y = 1|\mathrm{x}, \mathrm{w}) = \sigma(\mathrm{w}^T \mathrm{x} + \mathrm{b})$$

  où

- $$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- w : vecteur des poids .
- bs'appelle le biais.

Régression logistique : données binaires et courbe de probabilité



Pour chaque x, y suit une loi de Bernoulli de paramètre $\sigma(wx + b)$. Le biais b permet de décaler horizontalement la courbe.

Vraisemblance et log-vraisemblance

Pour n observations (x_i, y_i) indépendantes, la vraisemblance est :

$$L(\mathrm{w}, b) = \prod_{i=1}^{n} \sigma(\mathrm{w}^T \mathrm{x}_i + b)^{y_i} \big(1 - \sigma(\mathrm{w}^T \mathrm{x}_i + b)\big)^{1-y_i}$$

On maximise $\log L(\mathrm{w}, b)$:

$$\ell(\mathrm{w}, b) = \sum_{i=1}^{n} \big[y_i \log \sigma(\mathrm{w}^T \mathrm{x}_i + b) + (1 - y_i) \log\big(1 - \sigma(\mathrm{w}^T \mathrm{x}_i + b)\big)\big]$$

Problème de maximisation : $\max_{\mathrm{w}, b} \ell(\mathrm{w}, b)$.

Passage à une minimisation

Pour utiliser la descente de gradient, on minimise la *log-vraisemblance négative* :

$$J(\mathrm{w}, b) = -\ell(\mathrm{w}, b) = -\sum_{i=1}^{n} \big[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \big], \quad \hat{y}_i = \sigma(\mathrm{w}^T \mathrm{x}_i + b)$$

C'est l'entropie croisée.

Gradient

Le gradient de J par rapport à w et b est :

$$\nabla_{\mathrm{w}} J = \sum_{i=1}^{n} (\hat{y}_i - y_i)\mathrm{x}_i, \qquad \frac{\partial J}{\partial b} = \sum_{i=1}^{n} (\hat{y}_i - y_i)$$

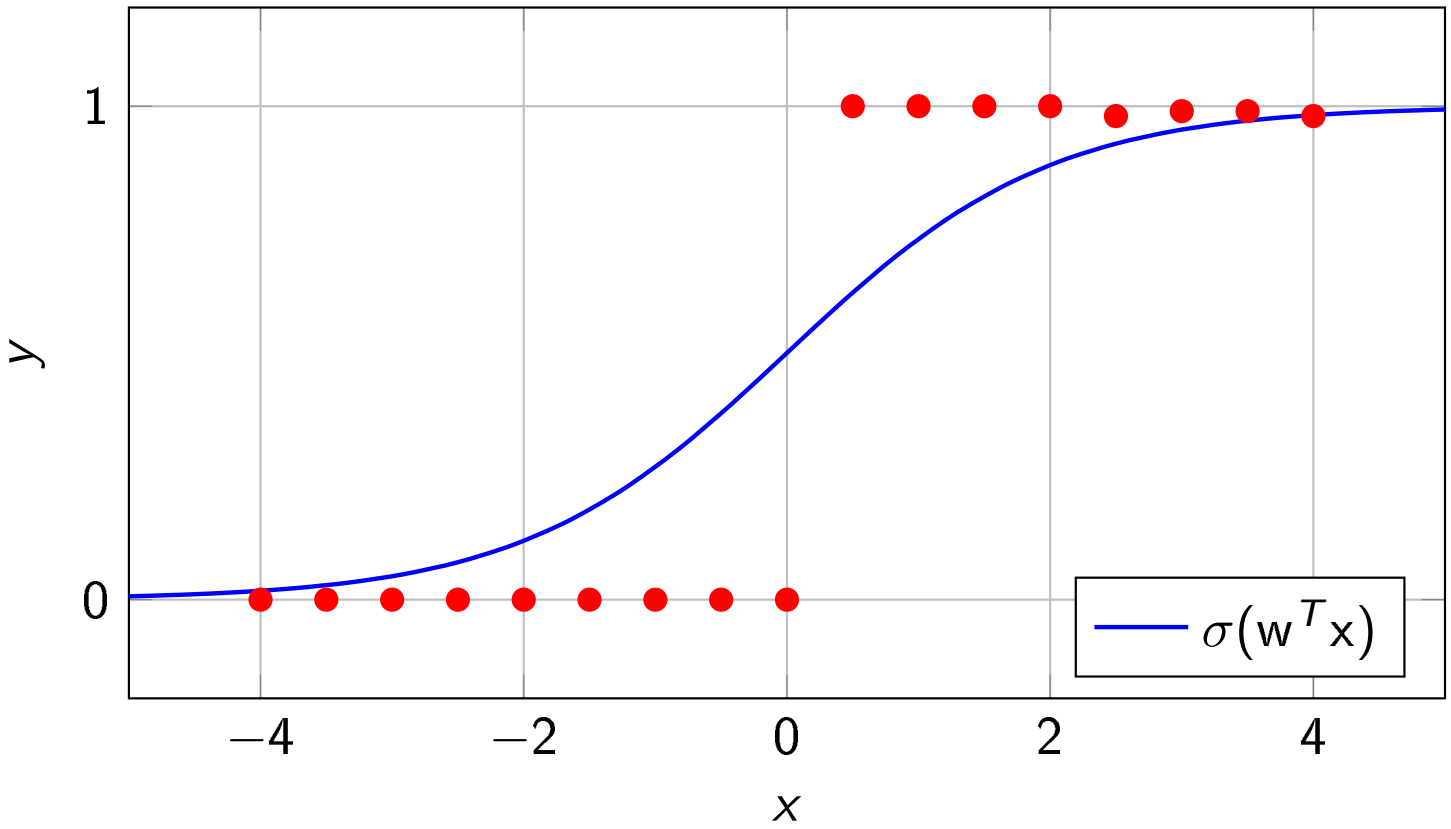
ou, sous forme moyenne :

$$\nabla_{\mathrm{w}} J = \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)\mathrm{x}_i, \quad \frac{\partial J}{\partial b} = \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)$$

La descente de gradient sur J :

$$\mathrm{w}_{k+1} = \mathrm{w}_k - \eta \nabla_{\mathrm{w}} J, \quad b_{k+1} = b_k - \eta \frac{\partial J}{\partial b}$$

Régression logistique : données binaires et courbe de probabilité



Pour chaque x, y suit une loi de Bernoulli de paramètre $\sigma(\mathrm{w}^T\mathrm{x})$.

Vraisemblance et log-vraisemblance

Pour n observations (x_i, y_i) indépendantes, la vraisemblance est :

$$L(\mathrm{w}) = \prod_{i=1}^{n} \sigma(\mathrm{w}^T \mathrm{x}_i)^{y_i} \big(1 - \sigma(\mathrm{w}^T \mathrm{x}_i)\big)^{1-y_i}$$

On maximise $\log L(\mathrm{w})$:

$$\ell(\mathrm{w}) = \sum_{i=1}^{n} \big[y_i \log \sigma(\mathrm{w}^T \mathrm{x}_i) + (1 - y_i) \log\big(1 - \sigma(\mathrm{w}^T \mathrm{x}_i)\big)\big]$$

Problème de maximisation : $\max_{\mathrm{w}} \ell(\mathrm{w})$.

Passage à une minimisation

Pour utiliser la descente de gradient, on minimise la *log-vraisemblance négative* :

$$J(\mathrm{w}) = -\ell(\mathrm{w}) = -\sum_{i=1}^{n} \big[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \big], \quad \hat{y}_i = \sigma(\mathrm{w}^T \mathrm{x}_i)$$

C'est l'entropie croisée.

Gradient

Le gradient de $J(\mathrm{w})$ par rapport à w est :

$$\nabla J(\mathrm{w}) = \sum_{i=1}^{n} (\hat{y}_i - y_i)\mathrm{x}_i$$

ou, sous forme moyenne :

$$\nabla J(\mathrm{w}) = \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)\mathrm{x}_i$$

La descente de gradient sur J :

$$\mathrm{w}_{k+1} = \mathrm{w}_k - \eta \nabla J(\mathrm{w}_k)$$

Références

- La fonction J est convexe (car l'entropie croisée avec sigmoïde est convexe).
- Descente de gradient (ou montée) converge vers l'unique minimum (maximum) pour un pas bien choisi.
- Pour de grands jeux de données, on utilise la descente de gradient stochastique (SGD) .

Références

- Boyd, S. et Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.
- Nocedal, J. et Wright, S. (2006). *Numerical Optimization*. Springer.
- Goodfellow, I., Bengio, Y., et Courville, A. (2016). *Deep Learning*. MIT Press.
- Polyak, B. (1987). *Introduction to Optimization*. Optimization Software.

l'Apprentissage

Supervisé vs Non Supervisé

Apprentissage Supervisé

- **Principe :** Apprendre à partir d'exemples étiquetés
- **Entrée :** (x, y) où y est la vérité terrain
- **Objectif :** Prédire y pour de nouvelles données x
- **Exemple :** Reconnaissance d'images (chat/chien)

Apprentissage Non Supervisé

- **Principe :** Trouver des structures cachées sans étiquettes
- **Entrée :** (x) seulement
- **Objectif :** Découvrir des groupes ou des régularités
- **Exemple :** Segmentation de clients (comportements similaires)

Métaphore

Supervisé = Examen avec corrigé (on apprend de ses erreurs)
Non supervisé = Puzzle sans image modèle (on cherche des motifs)

Apprentissage Supervisé vs Non Supervisé

Schéma comparatif

Supervisé

Données (x, y)

↓

Modèle

↓

Prédiction $\hat{y}$

Non Supervisé

Données (x)

↓

Modèle

↓

Patterns cachés

Algorithmes :

- Régression linéaire
- KNN
- Arbres de décision

Algorithmes :

- K-means
- PCA

Apprentissage supervisé

1) Méthode des K-plus proches voisins (KNN)

KNN pour classification

méthode des K-plus proches voisins (KNN)

Definition

La méthode des K-plus proches voisins (KNN) est un algorithme d'apprentissage supervisé qui prédit l'étiquette d'un point de test en examinant les étiquettes de ses K plus proches voisins dans l'ensemble d'entraînement. Pour la classification, on utilise un vote majoritaire ; pour la régression, on prend la moyenne des valeurs des voisins.

Caractéristiques principales

- **Non paramétrique** : Aucune hypothèse sur la distribution sous-jacente des données.
- **Paresseux (lazy learning)** : Pas de phase d'entraînement explicite ; tout le calcul est reporté à la prédiction.
- **Fondé sur la similarité** : La notion de proximité est essentielle.

Principe

Soit un ensemble d'entraînement $\mathcal{D} = \{(\mathrm{x}_i, y_i)\}_{i=1}^n$ où $\mathrm{x}_i \in \mathbb{R}^p$ est le vecteur de caractéristiques et y_i est l'étiquette (discrète pour la classification, continue pour la régression). Pour un nouveau point x_{test}, KNN procède en trois étapes :

1. Calculer les distances entre x_{test} et tous les x_i.
2. Sélectionner les K points ayant les plus petites distances.
3. Agréger leurs étiquettes :
   - **Classification** : $\hat{y} = \text{mode}(\{y_i\}_{i \in \mathcal{N}_K})$
   - **Régression** : $\hat{y} = \frac{1}{K} \sum_{i \in \mathcal{N}_K} y_i$

Exemple

Illustration : Un point vert à classer.
Avec K=3, voisins : 2 rouges, 1 bleu $\rightarrow$ classe rouge.
Avec K=5, voisins : 3 bleus, 2 rouges $\rightarrow$ classe bleue.

Figure – Influence du paramètre K sur la décision

Techniques de normalisation

KNN étant basé sur des distances, une caractéristique avec une grande échelle domine les autres.

1. **Min-Max scaling** :

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)} \in [0, 1]$$

2. **Standardisation (Z-score)** :

$$x' = \frac{x - \mu}{\sigma}$$

   où μ est la moyenne et σ l'écart-type.

Min-Max, Standardisation : sont des étapes de prétraitement, transforment les données en des valeurs comparables entre 0 et 1.

Algorithme KNN pour classification

Algorithm 1 Algorithme : KNN pour classification

Require: Ensemble d'entraînement $\mathcal{D}$, point de test $\mathrm{x}_{\mathrm{test}}$, entier K, métrique de distance d
Ensure: Étiquette prédite $\hat{y}$
  for each $(\mathrm{x}_i, y_i) \in \mathcal{D}$ **do**
    Calculer $d_i = d(\mathrm{x}_{\mathrm{test}}, \mathrm{x}_i)$
  end for
  Trier les indices par d_i croissant
  Sélectionner les K premiers indices : $\mathcal{N}$
  Compter les étiquettes dans $\mathcal{N}$
  $\hat{y} \leftarrow$ étiquette majoritaire
  return $\hat{y}$

Mesures de distance

Le choix de la distance est crucial pour KNN et autres algorithmes

Exemple

Distance de Minkowski pour des vecteurs $\mathrm{a}, \mathrm{b} \in \mathbb{R}^p$:

$$d(\mathrm{a}, \mathrm{b}) = \left(\sum_{i=1}^{p} |a_i - b_i|^q \right)^{1/q}$$

Valeur de q	**Nom**	**Formule**
$q = 1$	Manhattan (L_1)	$d = \sum_{i=1}^{p} \lvert a_i - b_i \rvert$
$q = 2$	Euclidienne (L_2)	$d = \sqrt{\sum_{i=1}^{p} (a_i - b_i)^2}$
$q \to \infty$	Tchebychev (L_∞)	$d = \max_i \lvert a_i - b_i \rvert$

Choix du paramètre K

- **Petit K** ($K = 1$) : Modèle complexe, surapprentissage (bruit).
- **Grand K** : Modèle simple, lisse, mais risque de sous-apprentissage.
- **Cas limites** : $K = n \rightarrow$ prédiction = classe majoritaire globale.

On utilise généralement la **validation croisée** (cross-validation) :

1. Diviser les données en k plis (folds : sous-ensembles de taille égale).
2. Pour chaque valeur candidate de K, entraîner sur $k - 1$ plis, évaluer sur le pli restant.
3. Choisir K minimisant l'erreur moyenne.

Variantes pondérées

On donne plus de poids aux voisins proches :

$$\hat{y} = \arg\max_{c} \sum_{i \in \mathcal{N}_K} w_i \cdot 1(y_i = c)$$

avec

$$w_i = \frac{1}{d(\mathrm{x}_{\mathrm{test}}, \mathrm{x}_i) + \epsilon}.$$

Utilisation de fonctions noyau (gaussien, par exemple) :

$$w_i = \exp\left(-\frac{d(\mathrm{x}_{\mathrm{test}}, \mathrm{x}_i)^2}{2\sigma^2}\right)$$

Apprentissag provisé

2) Arbre de décision

Definition

Un arbre de décision est un modèle prédictif structuré sous forme d'arbre binaire ou non. Chaque nœud interne représente un test sur une caractéristique, chaque branche correspond à un résultat du test, et chaque feuille (nœud terminal) donne une prédiction : classe majoritaire pour la classification, moyenne des valeurs pour la régression.

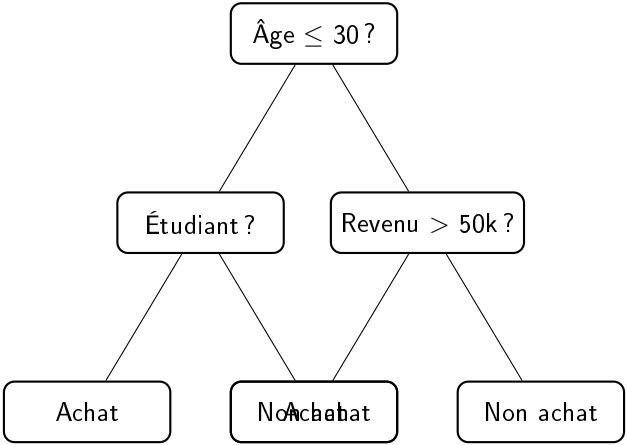


Figure – Exemple d'arbre de décision pour la prédiction d'achat

Structure d'un arbre de décision

- **Racine** : Nœud initial contenant toutes les données.
- **Nœud interne** : Effectue un test sur un attribut.
- **Branche** : Résultat d'un test.
- **Feuille** : Nœud terminal contenant la prédiction.
- **Profondeur** : Nombre maximum de nœuds de la racine à une feuille.

Principe de construction

L'algorithme construit l'arbre récursivement :

1. Si tous les exemples d'un nœud ont la même classe, arrêter.
2. Sinon, choisir le meilleur attribut pour diviser les données.
3. Partitionner les données selon les valeurs de l'attribut.
4. Répéter récursivement sur chaque sous-ensemble.

Critères de partition pour la classification

Soit un nœud contenant n exemples répartis en C classes, avec p_i la proportion d'exemples de classe i.
L'impureté mesure le désordre.

Impureté d'un nœud

Nœud PUR

10 Pommes
0 Poires

Impureté = 0

Nœud IMPUR

5 Pommes
5 Poires

Impureté = maximale

Formule générale

$$\text{Impureté} = f(p_1, p_2, ..., p_C) \quad \text{avec} \quad \sum p_i = 1$$

Plus les proportions sont déséquilibrées, plus le nœud est pur.

Les formules d'impureté : Entropie

Definition (Entropie de Shannon)

$$H(S) = -\sum_{i=1}^{C} p_i \log_2(p_i)$$

Avec $0 \log_2(0) = 0$. L'entropie est maximale ($\log_2 C$) quand toutes les classes sont équilibrées, nulle quand une classe domine.

Entropie faible (proche de 0) $\rightarrow$ nœud pur (presque tous de la même classe)
Entropie élevée (proche de 1) $\rightarrow$ nœud impur (classes bien mélangées)

Example

Pour un nœud avec 9 exemples : 5 "Oui", 4 "Non".

$$H = -\frac{5}{9}\log_2\frac{5}{9} - \frac{4}{9}\log_2\frac{4}{9} \approx 0.991$$

Formule de Gain

Definition (Gain d'information)

Pour une partition A avec v valeurs distinctes :

$$\text{Gain}(S, A) = H(S) - H_p(S)$$

où $H_p(S)$ est l'entropie pondérée est donnée par

$$H_p(S) = \sum_{v \in \text{Vals}(A)} \frac{|S_v|}{|S|} H(S_v)$$

On choisit l'attribut maximisant le gain.

ID3 (Iterative Dichotomiser 3)

Algorithm 2 ID3 - Construction d'arbre de classification

Require: Exemples S, attributs A
Ensure: Arbre de décision

```
if tous les exemples ont même classe c then
    return feuille étiquetée c
end if
if A est vide then
    return feuille avec classe majoritaire de S
end if
a* ← arg max_{a∈A} Gain(S, a)
Créer un nœud pour a*
for chaque valeur v de a* do
    S_v ← sous-ensemble de S avec a* = v
    Sous-arbre ← ID3(S_v, A \ {a*})
    Ajouter branche v → sous-arbre
end for
return arbre
```

Conclusion

Les arbres de décision sont des modèles fondamentaux en apprentissage automatique. Leur interprétabilité les rend précieux pour l'exploration de données et la prise de décision explicable. Bien qu'ils aient des limites (instabilité, surapprentissage), leurs extensions comme les forêts aléatoires et le gradient boosting en font des outils extrêmement puissants, souvent compétitifs avec les méthodes modernes.

Apprentissage non provisé

1) K-means

- Comprendre le problème de **clustering non supervisé**
- Formuler mathématiquement le problème K-means
- Étudier l'algorithme de Lloyd
- Analyser la convergence et la complexité
- Discuter les limites et extensions

Apprentissage non supervisé

On observe un ensemble de données :

$$X = \{x_1, \ldots, x_n\}, \quad x_i \in \mathbb{R}^d$$

Objectif : regrouper les données en K groupes homogènes.

Contrairement à l'apprentissage supervisé :

- Pas de labels
- Structure cachée à découvrir

Problème de clustering

On cherche une partition des clusters :

$$\{C_1, \ldots, C_K\}$$

Minimisant l'inertie intra-classe :

$$\min_{C_1,\ldots,C_K} \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

où

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

Idée : minimiser la variance intra-cluster.

Algorithme de Lloyd

Initialiser $\mu_1, \ldots, \mu_K$

Étape 1 : Affectation

$$C_k = \{x_i : \|x_i - \mu_k\|^2 \leq \|x_i - \mu_j\|^2, \forall j\}$$

Étape 2 : Mise à jour

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

Répéter jusqu'à convergence.

Convergence

$$\min_{\mu_1,\ldots,\mu_K} J = \sum_{i=1}^{n} \min_k \|x_i - \mu_k\|^2$$

À chaque itération :

$$J^{(t+1)} \leq J^{(t)}$$

Propriétés :

- Décroissance monotone
- Nombre fini de partitions
- Convergence vers un minimum local

Pourquoi la moyenne ?

Pour un cluster fixé C_k :

$$\min_{\mu} \sum_{x_i \in C_k} \|x_i - \mu\|^2$$

Condition du premier ordre :

$$\frac{\partial}{\partial \mu} \sum_{x_i \in C_k} \|x_i - \mu\|^2 = 0$$

Implique :

$$\mu = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

Donc la moyenne minimise l'erreur quadratique.

Complexité

À chaque itération :

$$O(nKd)$$

où :

- n = nombre de points
- K = nombre de clusters
- d = dimension

Problème global : NP-difficile.

Exemple avec poids et taille

La variance intra-cluster mesure à quel point les points d'un même groupe sont serrés ou dispersés.

Groupe de 3 personnes

Nom	Poids (kg)	Taille (cm)
Alice	60	160
Bob	65	165
Charlie	70	170

Centre μ

$$\mu = \left(\frac{60+65+70}{3}, \frac{160+165+170}{3}\right)$$

$$\mu = (65, 165)$$

Calcul des distances au carré

- Alice : $(60-65)^2 + (160-165)^2 = 25 + 25 = 50$
- Bob : $(65-65)^2 + (165-165)^2 = 0 + 0 = 0$
- Charlie : $(70-65)^2 + (170-165)^2 = 25 + 25 = 50$

Résultat

$$\text{Variance intra closter} = \frac{50+0+50}{3} = \frac{100}{3} \approx 33,33$$

Extensions

- K-medoids
- Spectral clustering
- Gaussian Mixture Models (EM)
- Mini-batch K-means

2)Réduction de dimension

- Idée de la PCA
- Projection
- Variance expliquée

Pourquoi réduire la dimension ?

On observe :

$$x_i \in \mathbb{R}^d, \quad d \text{ grand}$$

Problèmes :

- Malédiction de la dimension
- Surapprentissage
- Coût computationnel élevé
- Redondance des variables

Objectif :

$$\text{Trouver } y_i \in \mathbb{R}^r, \quad r \ll d$$

en conservant un maximum d'information.

Idée centrale de la PCA

Principe :
Projeter les données sur une direction maximisant la variance.

Données centrées :

$$\tilde{x}_i = x_i - \bar{x}$$

On cherche un vecteur u tel que :

$$\max_{\|u\|=1} \sum_{i=1}^{n} (u^T \tilde{x}_i)^2$$

Maximiser la variance projetée.

Formulation matricielle

Matrice des données centrées :

$$X \in \mathbb{R}^{n \times d}$$

Matrice de covariance :

$$\Sigma = \frac{1}{n} X^T X$$

La vriance :

$$Vra(u) = u^T \Sigma u$$

Le problème devient :

$$\max_{\|u\|=1} Var(u)$$

Problème de Rayleigh.

Solution : vecteur propre dominant

Résultat fondamental :

$$\max_{\|u\|=1} u^T \Sigma u$$

est atteint pour :

$$u = \text{vecteur propre associé à } \lambda_{\max}$$

où :

$$\Sigma u = \lambda u$$

La variance maximale est :

$$Var = \lambda_{\max}$$

Composantes principales multiples

Pour r dimensions :

$$\max_{U^T U = I_r} \mathrm{Tr}(U^T \Sigma U)$$

Solution :

$$U = [u_1, \ldots, u_r]$$

où u_k sont les vecteurs propres associés aux r plus grandes valeurs propres.

Projection des données

Projection :

$$Y = XU$$

avec :

$$Y \in \mathbb{R}^{n \times r}$$

Chaque ligne :

$$y_i = U^T x_i$$

On obtient une représentation de dimension réduite.

Variance expliquée

Si :

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d$$

Alors la variance totale :

$$\sum_{j=1}^{d} \lambda_j$$

Variance expliquée par les r premières composantes :

$$Var_e x = \frac{\sum_{j=1}^{r} \lambda_j}{\sum_{j=1}^{d} \lambda_j}$$

La variance expliquée mesure la quantité d'information conservée lorsqu'on projette les données sur un nombre réduit r de composantes principales,

Résumé

- PCA = projection linéaire
- Maximise la variance projetée
- Basée sur les valeurs propres
- Réduction optimale en norme Frobenius
- Outil fondamental en learning

Le jeu de données Iris

Description

- 150 fleurs d'iris
- 3 espèces : Setosa, Versicolor, Virginica
- 4 mesures par fleur :

Longueur sépale (cm)
Largeur sépale (cm)
Longueur pétale (cm)
Largeur pétale (cm)

Objectif

Visualiser ces données 4D sur un graphique 2D en perdant le moins d'information possible.

Problème : 4 dimensions, difficile à visualiser

Données brutes (4 variables)

	Sépale L	**Sépale l**	**Pétale L**	**Pétale l**
Iris 1	5,1	3,5	1,4	0,2
Iris 2	4,9	3,0	1,4	0,2
...	...	...	...	...
Iris 150	5,9	3,0	5,1	1,8

Question

Comment représenter ces 150 points dans un **graphique en 2D** qu'on peut regarder ?

Réponse : L'ACP va projeter ces 4 dimensions sur les 2 axes qui maximisent la variance.

Que donne l'ACP sur Iris ?

Valeurs propres (variances)

$$\lambda_1 \approx 4,23 \quad (72,9\%)$$

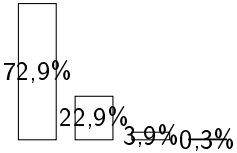
$$\lambda_2 \approx 0,24 \quad (22,9\%)$$

$$\lambda_3 \approx 0,08 \quad (3,9\%)$$

$$\lambda_4 \approx 0,02 \quad (0,3\%)$$

Variance cumulée

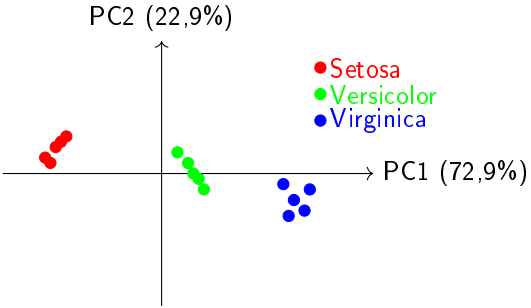
PC	Cumulé
PC1	72,9%
PC2	95,8%
PC3	99,7%
PC4	100%



Interprétation

Les **2 premières composantes** (PC1 + PC2) expliquent 95,8% de la variance totale. → On peut passer de 4D à 2D en perdant seulement 4,2% d'information !

Le graphe qu'on obtient



Ce qu'on observe

- Les Setosa sont bien séparées des autres (à gauche)
- Versicolor et Virginica sont proches mais séparables
- Les 3 espèces forment des groupes distincts !

Évaluation

- 30% TP
- 70% Examen final